

11. Container With Most Water Medium

Acceptance: 59.3% | [LeetCode](#)

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are `(i, 0)` and `(i, height[i])`.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.

Example 1:

```
**Input:** height = [1,8,6,2,5,4,8,3,7]
**Output:** 49
**Explanation:** The above vertical lines are
```

represented by array `[1,8,6,2,5,4,8,3,7]`. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

```
**Input:** height = [1,1]
**Output:** 1
```

Constraints:

- `n == height.length`
- `2 <= n <= 105`
- `0 <= height[i] <= 104`

Claude Opus Solution

Container With Most Water

1. Intuition

The area of water between two lines `i` and `j` (where `i < j`) is:

```
area = min(height[i], height[j]) * (j - i)
          ↑ height             ↑ width
```

The water level is limited by the **shorter** of the two lines (you can't slant the container).

Brute force would check all pairs in $O(n^2)$. But we can do better with a key observation:

Key Insight: Start with the widest possible container (leftmost and rightmost lines). The width is maximized. To find a better area, we must increase the height of the container. Since the area is bounded by the *shorter* line, moving the **taller** line inward can never help (we lose width and the height stays capped by the shorter line). So we always move the **shorter** line inward, hoping to find a taller line that compensates for the lost width.

This is the classic **Two Pointers** pattern (converging from both ends).

2. Approach (Pseudocode)

1. Set `left = 0`, `right = n - 1`
2. Set `max_area = 0`
3. While `left < right`:
 - a. Compute current area = `min(height[left], height[right]) * (right - left)`
 - b. Update `max_area`
 - c. Move the pointer at the **SHORTER** line inward:
 - if `height[left] < height[right]`: `left++`
 - else: `right--`
4. Return `max_area`

Why is it safe to discard the shorter line? Suppose `height[left] < height[right]`. Any container using `left` with some inner line `k` (`left < k < right`) has: - smaller width (`k - left < right - left`) - height still capped at most by `height[left]`

So no future container with `left` can beat the current one. We can safely discard `left`.

3. Python Solution

```
from typing import List
```

```

class Solution:
    def maxArea(self, height: List[int]) -> int:
        # Two pointers starting at the widest container
        left, right = 0, len(height) - 1
        max_area = 0

        # Shrink the window until the pointers meet
        while left < right:
            # Width between the two chosen lines
            width = right - left

            # The water level is limited by the SHORTER line
            current_height = min(height[left], height[right])

            # Update the best area seen so far
            max_area = max(max_area, current_height * width)

            # Move the pointer at the shorter line inward.
            # Moving the taller line could only reduce width without
            # raising the (capped) height, so we discard the shorter one.
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1

        return max_area

```

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	Each iteration moves exactly one pointer inward. The pointers traverse the array at most once before meeting.
Space	$O(1)$	Only a constant number of variables (<code>left</code> , <code>right</code> , <code>max_area</code>) are used.

5. Alternative Approaches

A. Brute Force — $O(n^2)$ time, $O(1)$ space

Check every pair (`i` , `j`) and compute the area directly.

```

max_area = 0
for i in range(n):
    for j in range(i + 1, n):
        max_area = max(max_area, min(height[i], height[j]) * (j - i))
return max_area

```

Simple but too slow for $n = 10^5$ ($\approx 5 \times 10^9$ operations).

B. Two Pointers (the optimal solution above) — $O(n)$ time, $O(1)$ space

The accepted approach. It's the standard answer.

Note on the reference solution: It uses `while left != right` with `>=` to decide movement, which is functionally equivalent. The version above uses `left < right` and `<` for clarity — both are correct. A minor optimization (sometimes added) is to *skip* lines that aren't taller than the current shorter bound, but it doesn't change the asymptotic complexity.

Pattern Takeaway

This problem is a textbook example of the "**converging two pointers**" pattern. Whenever you're optimizing over pairs in an array and you can prove that moving one pointer never loses the optimal answer, two pointers turns $O(n^2)$ into $O(n)$.

Codex Solution

1. Intuition

For two lines at indices `left` and `right` , the water area is:

```
area = min(height[left], height[right]) * (right - left)
```

The width is `right - left`, and the height is limited by the shorter line.

A brute force solution checks every pair, but that takes $O(n^2)$ time.

The key insight is:

Start with the widest possible container, then move the pointer at the shorter line inward.

Why?

Suppose:

```
height[left] < height[right]
```

The current area is limited by `height[left]`.

If we move `right` inward, the width decreases, but the limiting height is still at most `height[left]`, because the left line did not change. So moving `right` cannot produce a better area with this same `left`.

Therefore, the only chance to find a bigger area is to move `left` inward and hope for a taller left line.

This is the classic **two pointers** pattern.

2. Approach

Pseudocode

```
initialize left = 0
initialize right = n - 1
initialize max_area = 0

while left < right:
    width = right - left
    current_height = min(height[left], height[right])
    current_area = width * current_height

    update max_area if current_area is larger

    if height[left] < height[right]:
        move left pointer rightward
    else:
        move right pointer leftward

return max_area
```

3. Python Solution

```
from typing import List

class Solution:
    def maxArea(self, height: List[int]) -> int:
        # Start with the widest possible container.
        left = 0
        right = len(height) - 1

        # Stores the best area found so far.
        max_area = 0

        # Continue until the two pointers meet.
        while left < right:
            # Width is the horizontal distance between the two lines.
            width = right - left

            # The container height is limited by the shorter line.
            container_height = min(height[left], height[right])

            # Compute the current container area.
            current_area = width * container_height
```

```
# Update the maximum area if this container is better.
max_area = max(max_area, current_area)

# Move the pointer at the shorter line.
#
# Reason:
# The shorter line limits the current area.
# Since moving either pointer reduces width, we need a chance
# to find a taller limiting line. Therefore, move the shorter side.
if height[left] < height[right]:
    left += 1
else:
    right -= 1

return max_area
```

4. Complexity Analysis

Time Complexity: $O(n)$

Each pointer moves at most n times total.

- `left` only moves to the right.
- `right` only moves to the left.
- Every iteration moves one pointer.

Therefore, the total number of iterations is at most $n - 1$.

Space Complexity: $O(1)$

We only use a few variables:

```
left, right, max_area, width, container_height, current_area
```

No extra data structures are needed.

5. Alternative Approaches

Brute Force

Check every possible pair of lines.

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        max_area = 0

        for i in range(len(height)):
            for j in range(i + 1, len(height)):
                width = j - i
                container_height = min(height[i], height[j])
                max_area = max(max_area, width * container_height)

        return max_area
```

Complexity

```
Time:  $O(n^2)$ 
Space:  $O(1)$ 
```

This works logically, but it is too slow for $n \leq 10^5$.

The optimized two-pointer solution is preferred.